

Linear Regression

Perceptron

Linear Regression

Loss function L is measure of our prediction error:

$$L(\mathbf{w}) = \sum_{i=1}^n (x_i^T \mathbf{w} - y_i)^2 = \min_{\mathbf{w}} \|\mathbf{X}\mathbf{w} - \mathbf{y}\|_2^2$$

$$\nabla L(\mathbf{w}) = 2\mathbf{X}^T (\mathbf{X}\mathbf{w} - \mathbf{y}) = 0$$

$$\mathbf{X}^T \mathbf{X} \mathbf{w} = \mathbf{X}^T \mathbf{y}$$

$$\mathbf{w}_{OLS}^* = (\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T \mathbf{y}$$

The main problem: computational cost

Time complexity (without advanced algorithms): $O(d^2 N + d^3)$, $\mathbf{X}: N \times d$

Linear Regression: Gradient Descent

The Mean Squared Error $\frac{1}{N} \|\mathbf{X}\mathbf{w} - \mathbf{y}\|_2^2$ is smooth and convex

➤ We can use iterative methods

$$L_{MSE}(g(\mathbf{X}), \mathbf{y}) = \frac{1}{N} \|\mathbf{X}\mathbf{w} - \mathbf{y}\|_2^2$$

$$\nabla_{\mathbf{w}} L_{MSE}(g(\mathbf{X}), \mathbf{y}) = \frac{2}{N} \mathbf{X}^T (\mathbf{X}\mathbf{w} - \mathbf{y})$$

Gradient descent:

$$\mathbf{w}_{n+1} = \mathbf{w}_n - \alpha \nabla_{\mathbf{w}} L_{MSE}(g(\mathbf{X}), \mathbf{y})$$

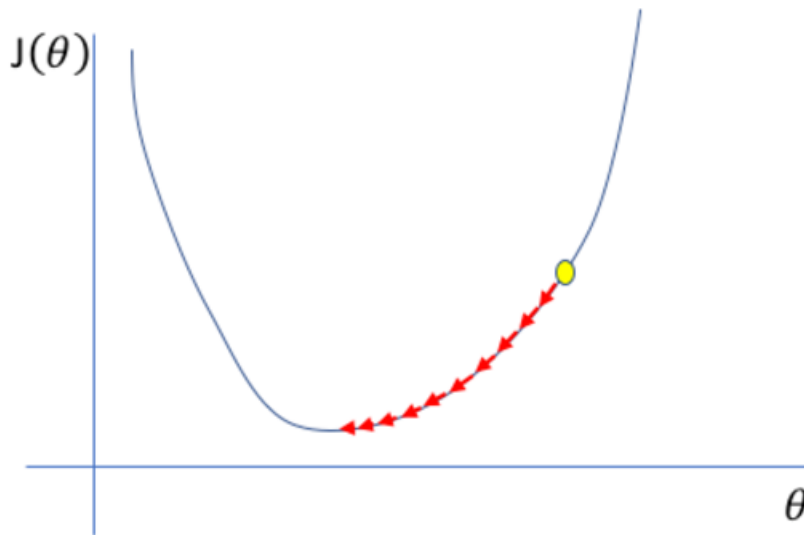
Time complexity: $O(NdK)$, K is number of steps

Gradient Descent: Properties

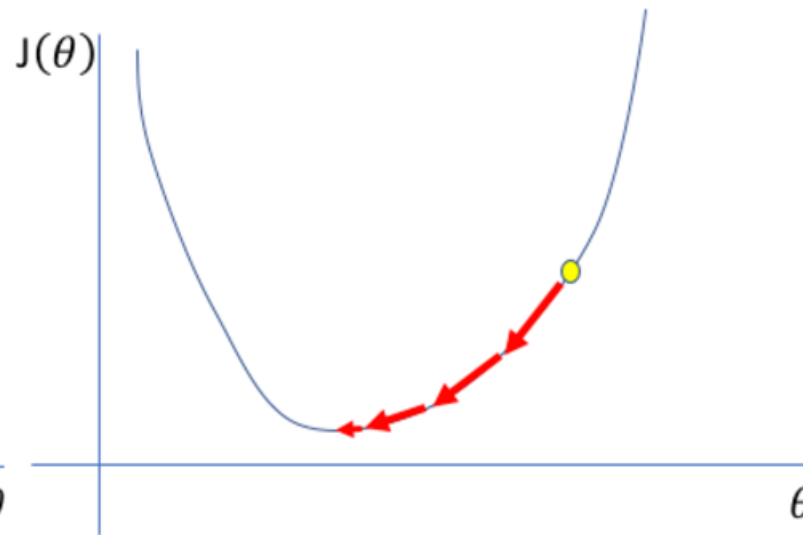
In our context:

- Depends on the initial point, but quite small (because it is a convex problem)
- Depends significantly on the condition number of $\mathbf{X}$
- Depends on the α

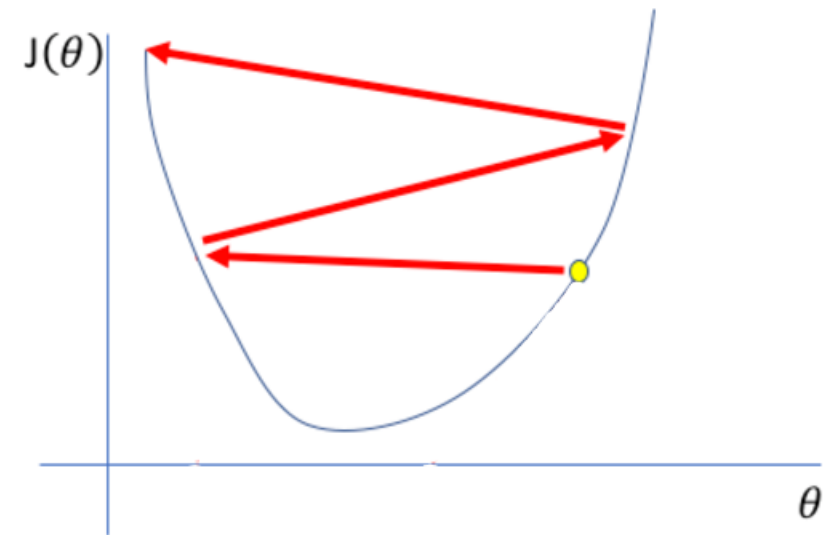
Learning rate α : too low



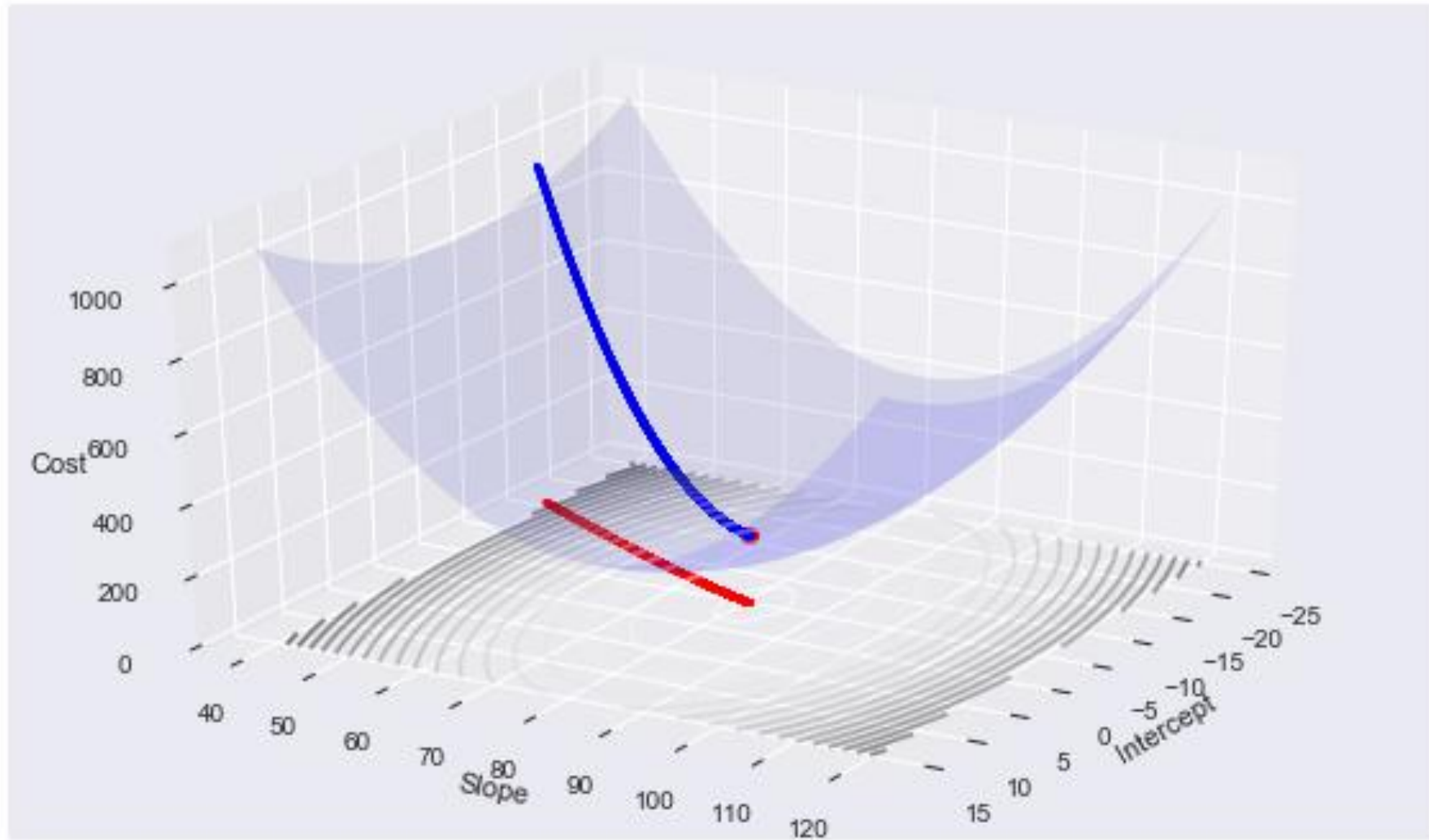
just right



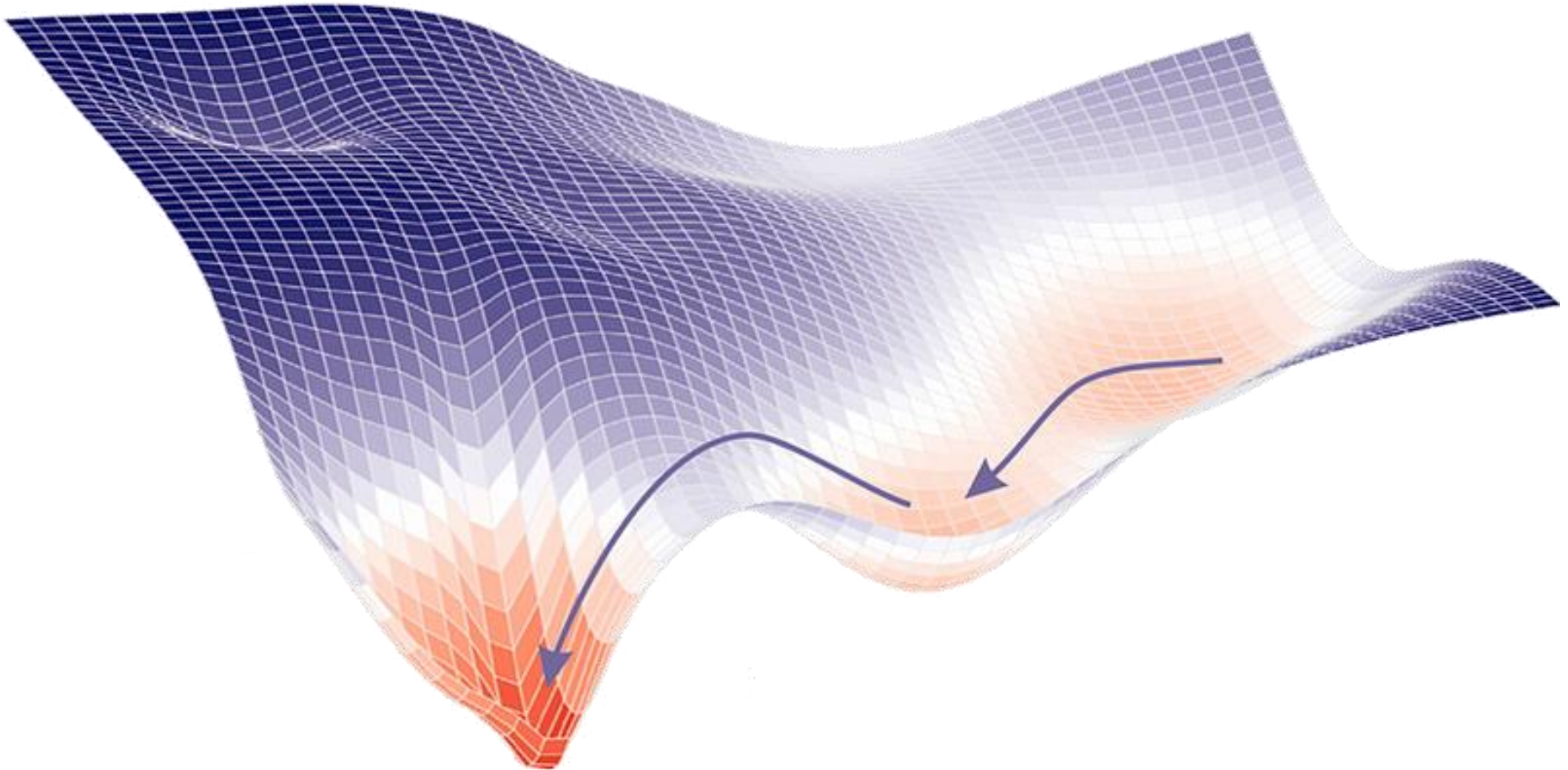
too high



Visualization: MSE for Linear Regression

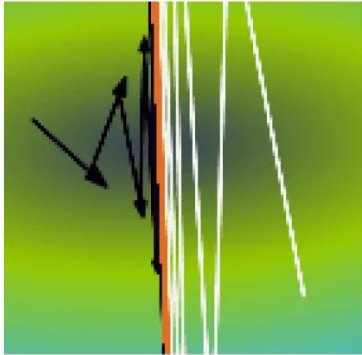


Visualization: More Complex Surface

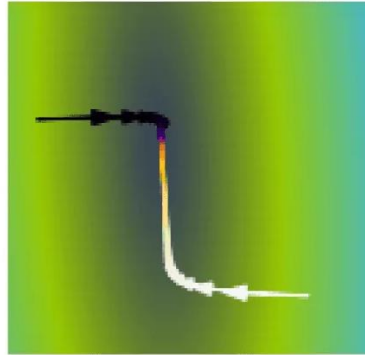


Visualization: Condition number and LR

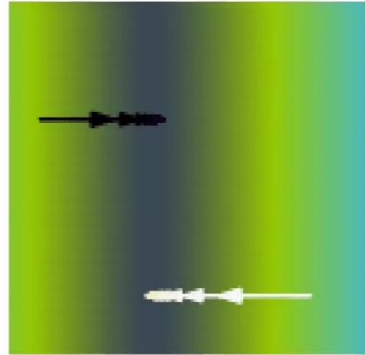
Round, $\alpha = 0.45$



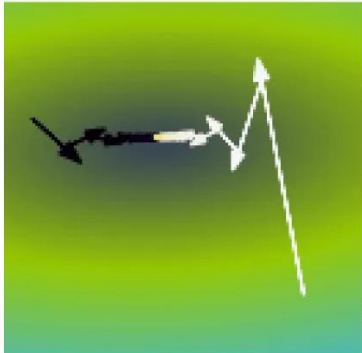
Elliptic, $\alpha = 0.45$



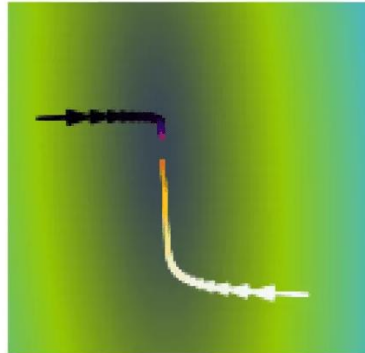
Stripe-like, $\alpha = 0.45$



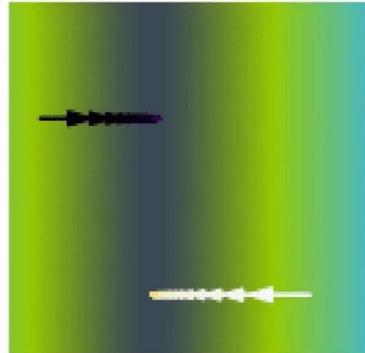
Round, $\alpha = 0.25$



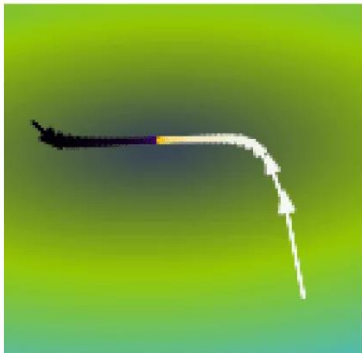
Elliptic, $\alpha = 0.25$



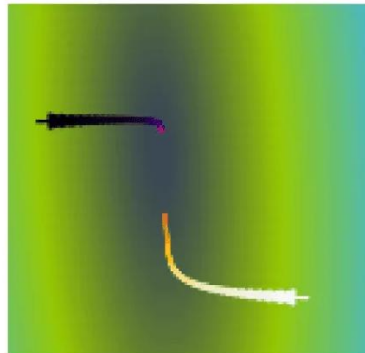
Stripe-like, $\alpha = 0.25$



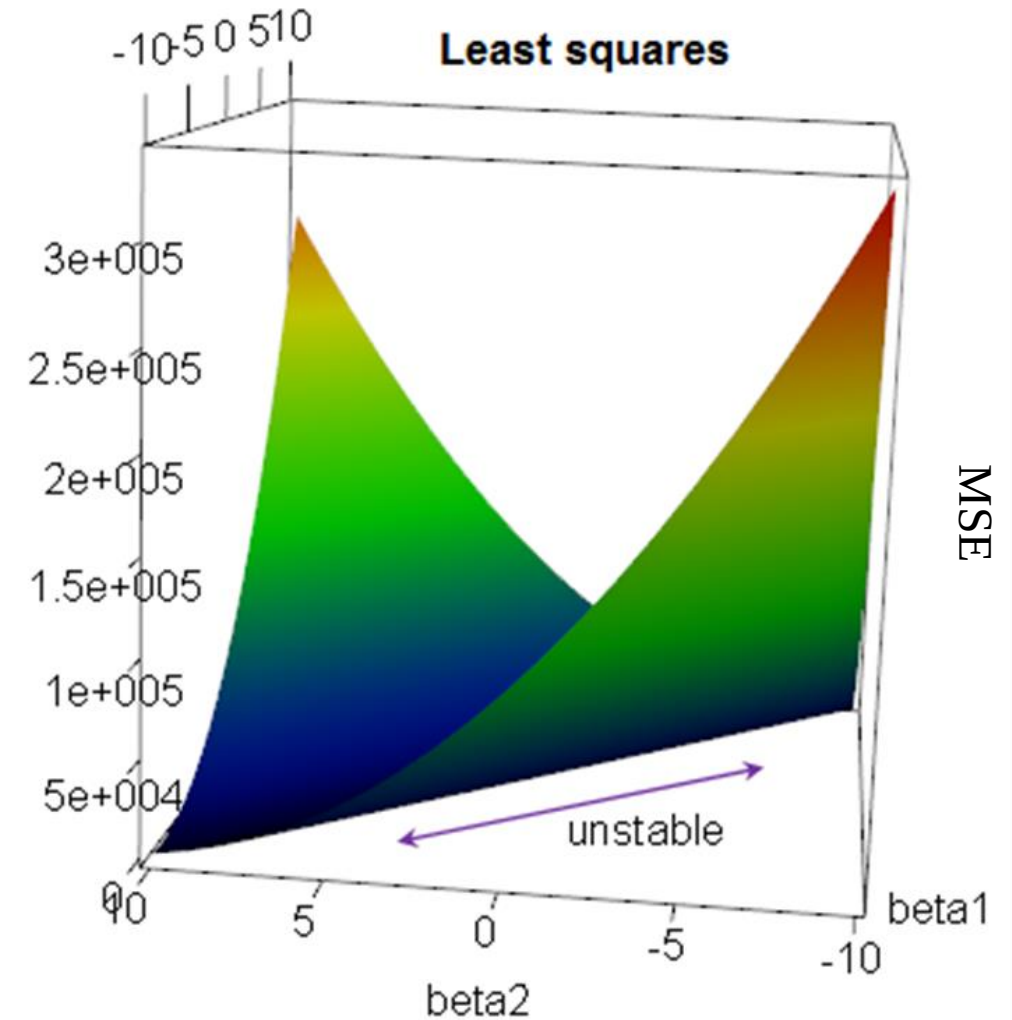
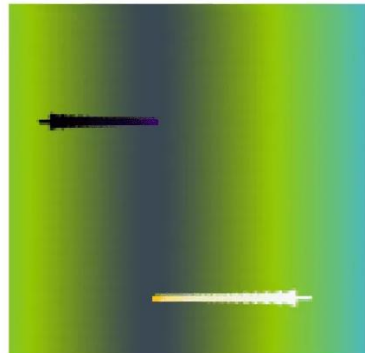
Round, $\alpha = 0.1$



Elliptic, $\alpha = 0.1$



Stripe-like, $\alpha = 0.1$



Ridge Regression: Gradient Descent

$$L_{RIDGE}(g(\mathbf{X}), \mathbf{y}) = \frac{1}{N} \|\mathbf{X}\mathbf{w} - \mathbf{y}\|_2^2 + \lambda \|\mathbf{w}'\|_2^2$$

$$\nabla_{\mathbf{w}} L_{RIDGE}(g(\mathbf{X}), \mathbf{y}) = \frac{2}{N} \mathbf{X}^T (\mathbf{X}\mathbf{w} - \mathbf{y}) + 2\lambda \mathbf{w}'$$

$\|\mathbf{w}'\|_2^2$ — L_2 regularization

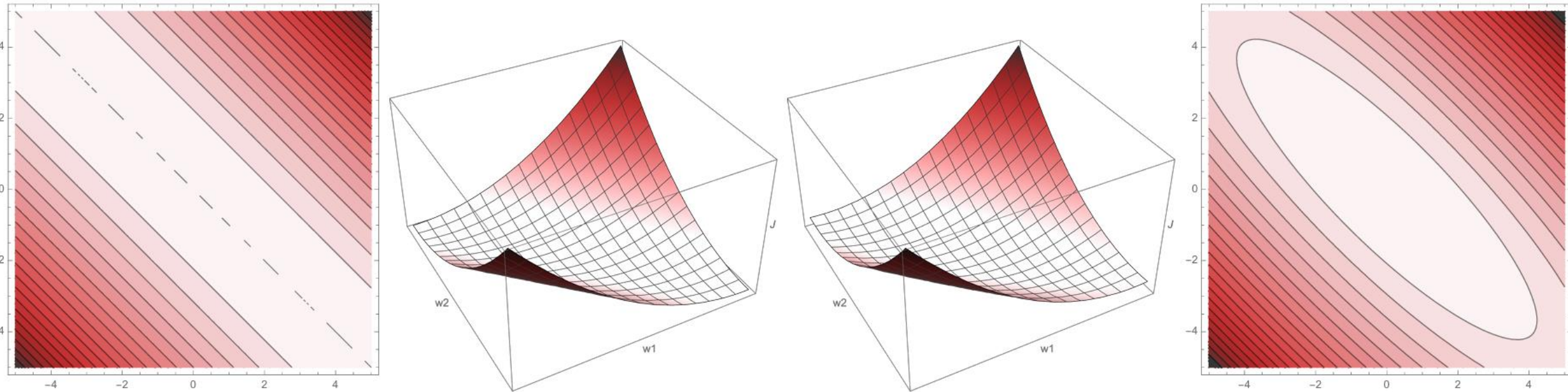
Gradient descent step:

$$\mathbf{w}_{n+1} = \mathbf{w}_n - \alpha \nabla_{\mathbf{w}} L_{RIDGE}(g(\mathbf{X}), \mathbf{y}) = \mathbf{w}_n - \alpha \frac{2}{N} \mathbf{X}^T (\mathbf{X}\mathbf{w} - \mathbf{y}) - 2\alpha \lambda \mathbf{w}'$$

$\mathbf{w}'$ — is a $\mathbf{w}$ without fictive dimension (for a bias)

(If we have normalized data then we can penalize the $\mathbf{w}$ fully)

Ridge Regression: Another Visualization



Ill-posed problem: large weights imply that tiny changes in x can cause huge changes in y

“Regularization” implies that we are turning an ill-posed problem into a well-posed problem

Feature Subset Selection

- Regression works good if $n > d$
- All features increase variance, but not all features reduce bias

Idea: identify poorly predictive features, ignore them (weight zero)

Corollary:

- Less overfitting, smaller test error
- Inference
- Easier to interpret

Ideal algorithm: best subset selection. Try all $2^d - 1$ nonempty subset of features.

Cons: combinatorically hard

Heuristics (often work well): forward/backward stepwise selection, $O(d^2)$ instead $O(2^d)$

Not ideal: we can't find the best n -feature model if we couldn't find the best $(n - 1)$ -feature

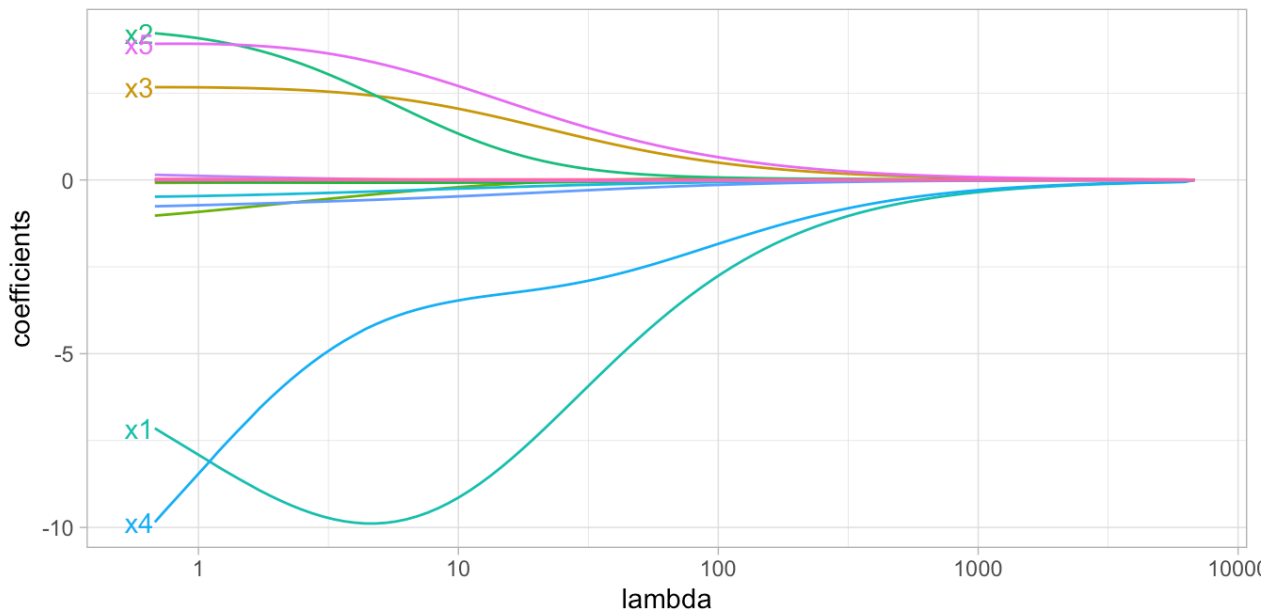
LASSO:

Least absolute shrinkage and selection operator

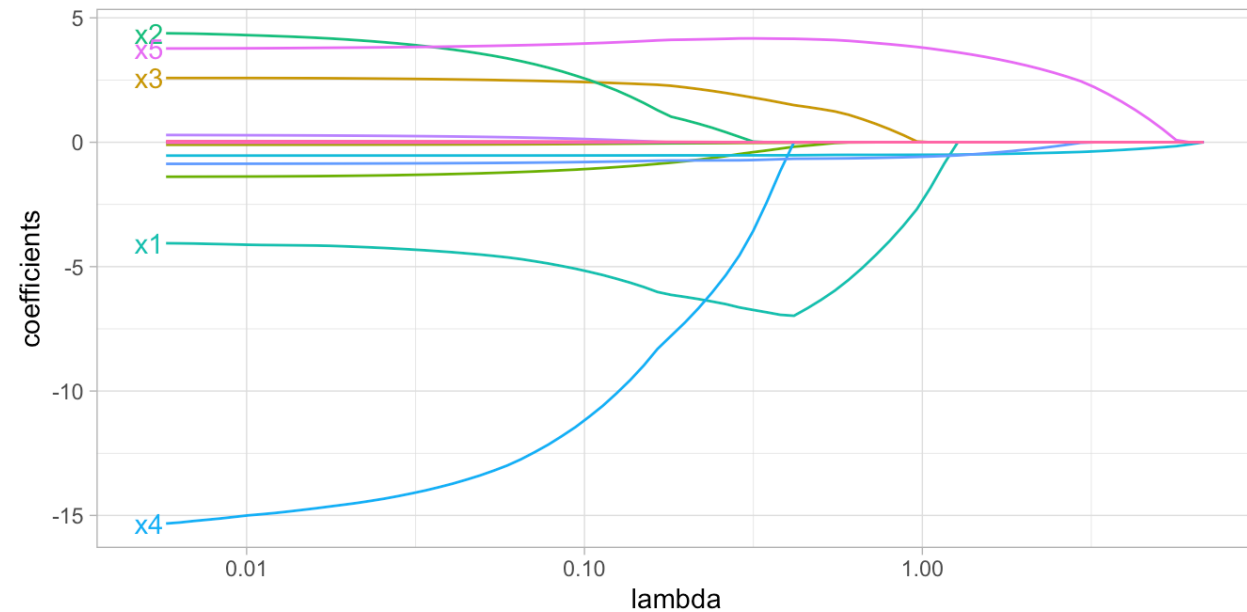
$$L_{LASSO}(g(\mathbf{X}), \mathbf{y}) = \frac{1}{N} \|\mathbf{X}\mathbf{w} - \mathbf{y}\|_2^2 + \lambda \|\mathbf{w}'\|_1$$

$\|\mathbf{w}\|_1 = \sum_{i=1}^d |w_i|$ – L_1 regularization

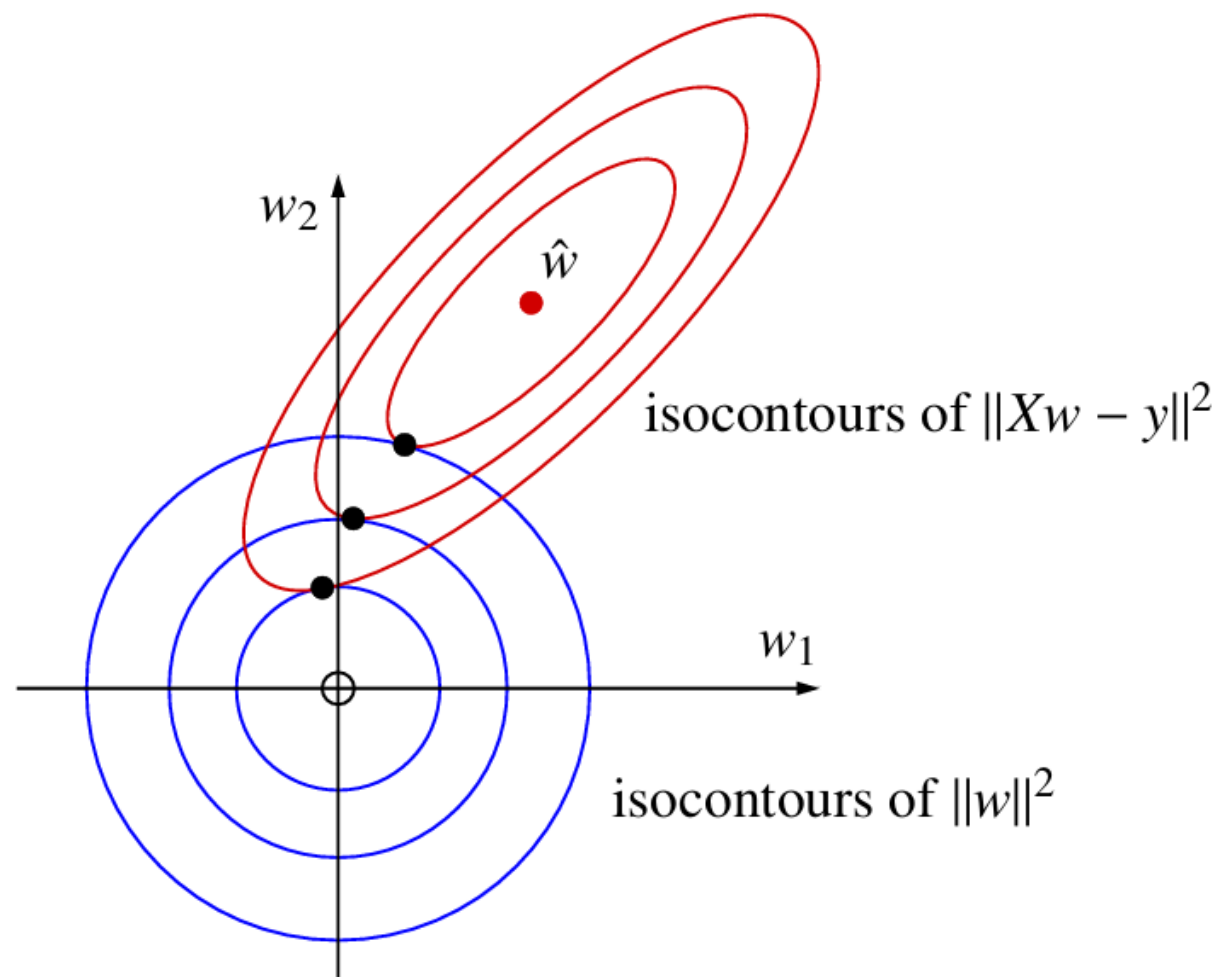
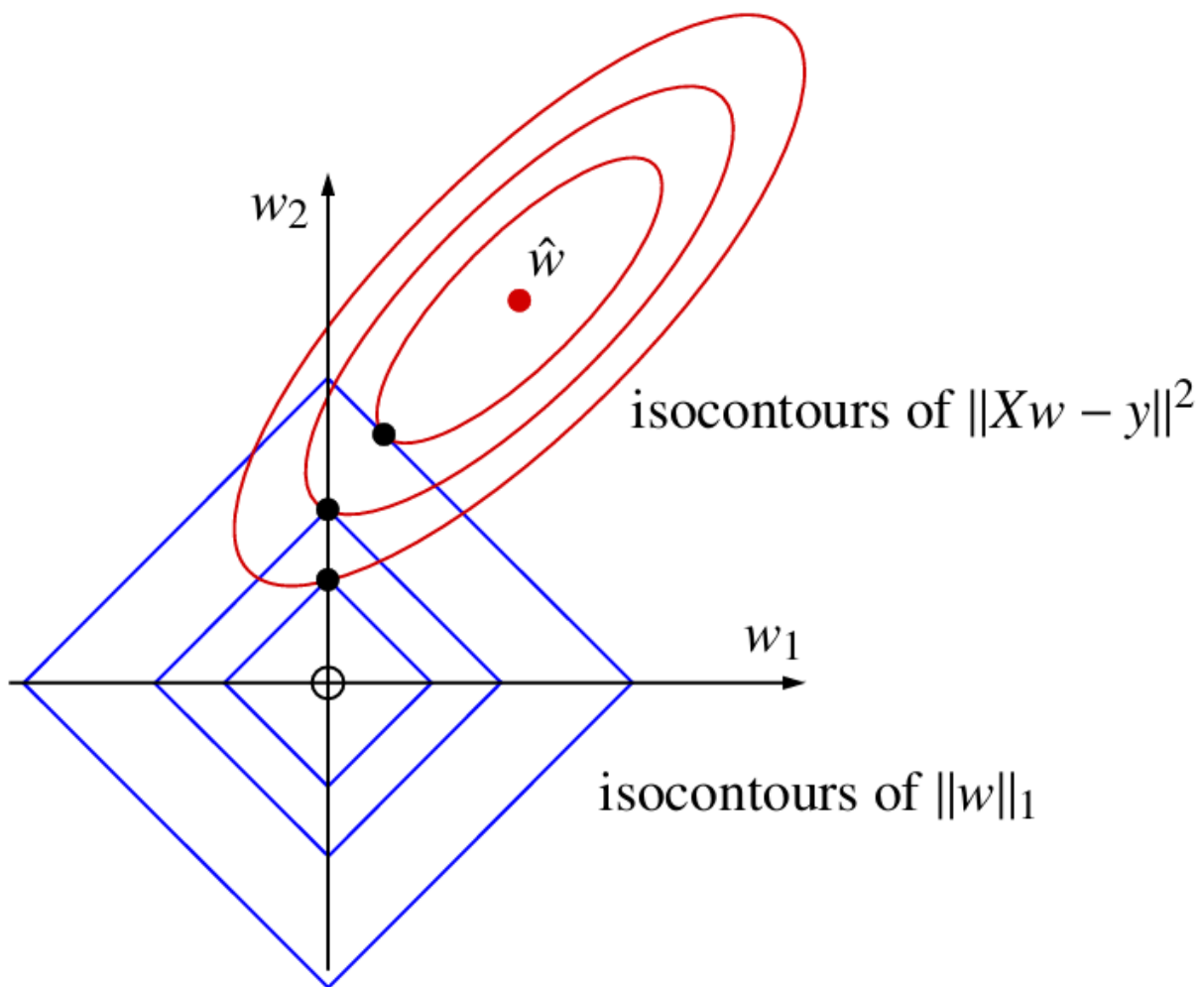
Ridge



Lasso

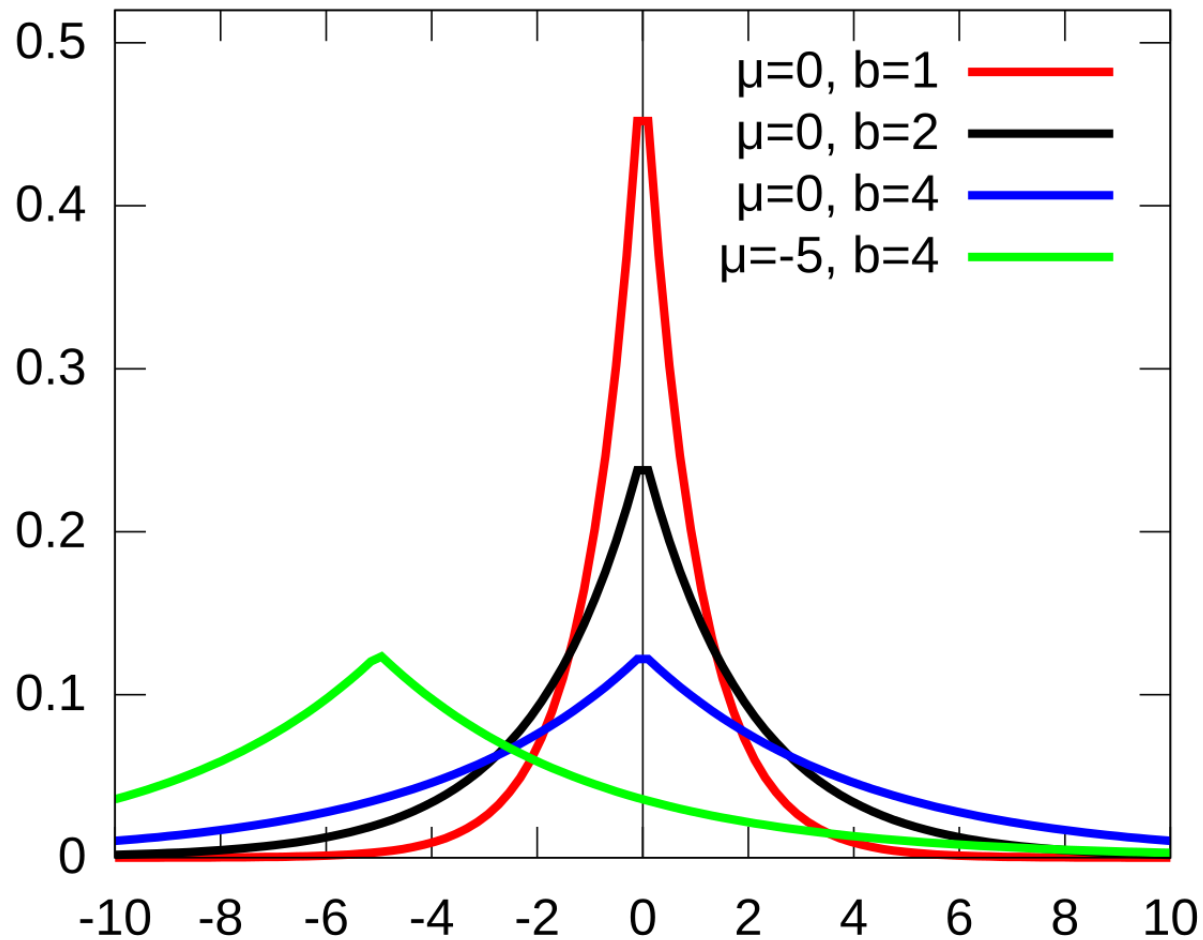


LASSO: Geometric Interpretation



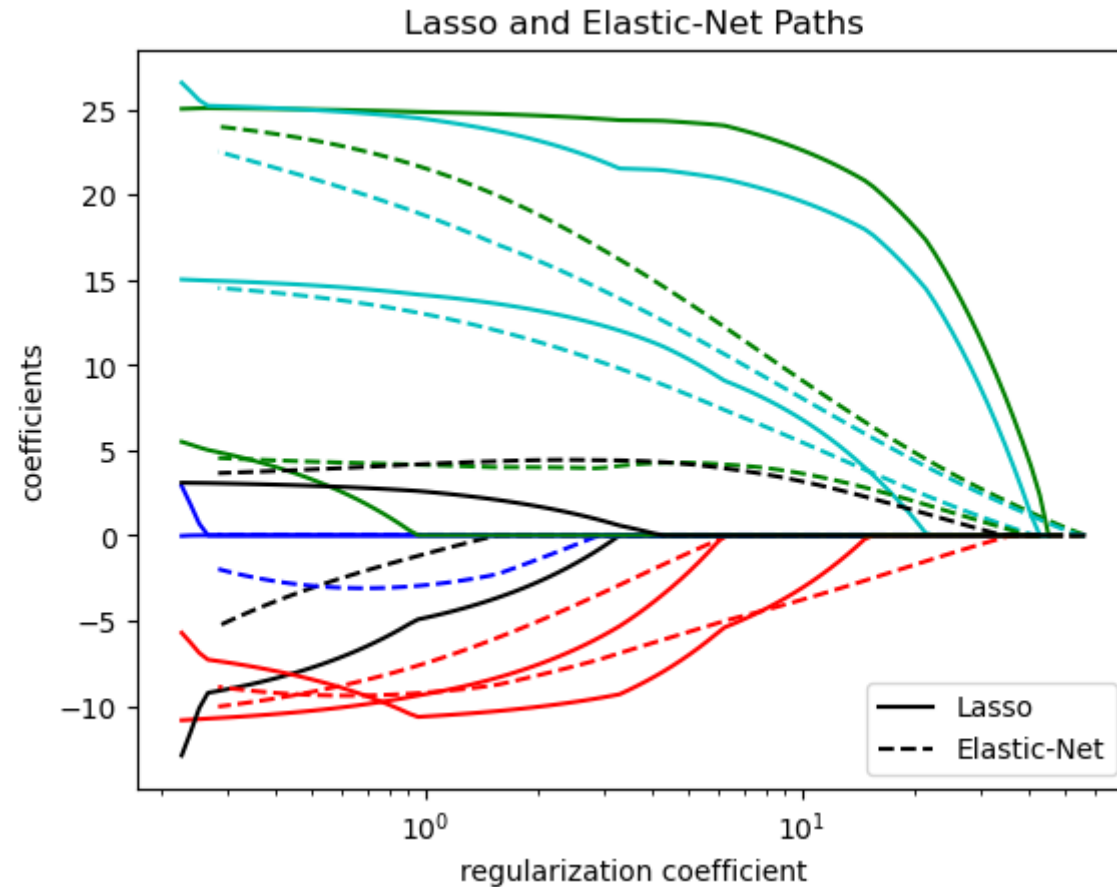
LASSO: Bayesian Interpretation

Prior: Laplace distribution



Elastic Net

$$L_{ELASTIC}(g(\mathbf{X}), \mathbf{y}) = \frac{1}{N} \|\mathbf{X}\mathbf{w} - \mathbf{y}\|_2^2 + \lambda(1 - l_{RATIO})\|\mathbf{w}'\|_2^2 + \lambda \cdot l_{RATIO}\|\mathbf{w}'\|_1$$



Coordinate Descent and Least Angle Regression

Coordinate descent:

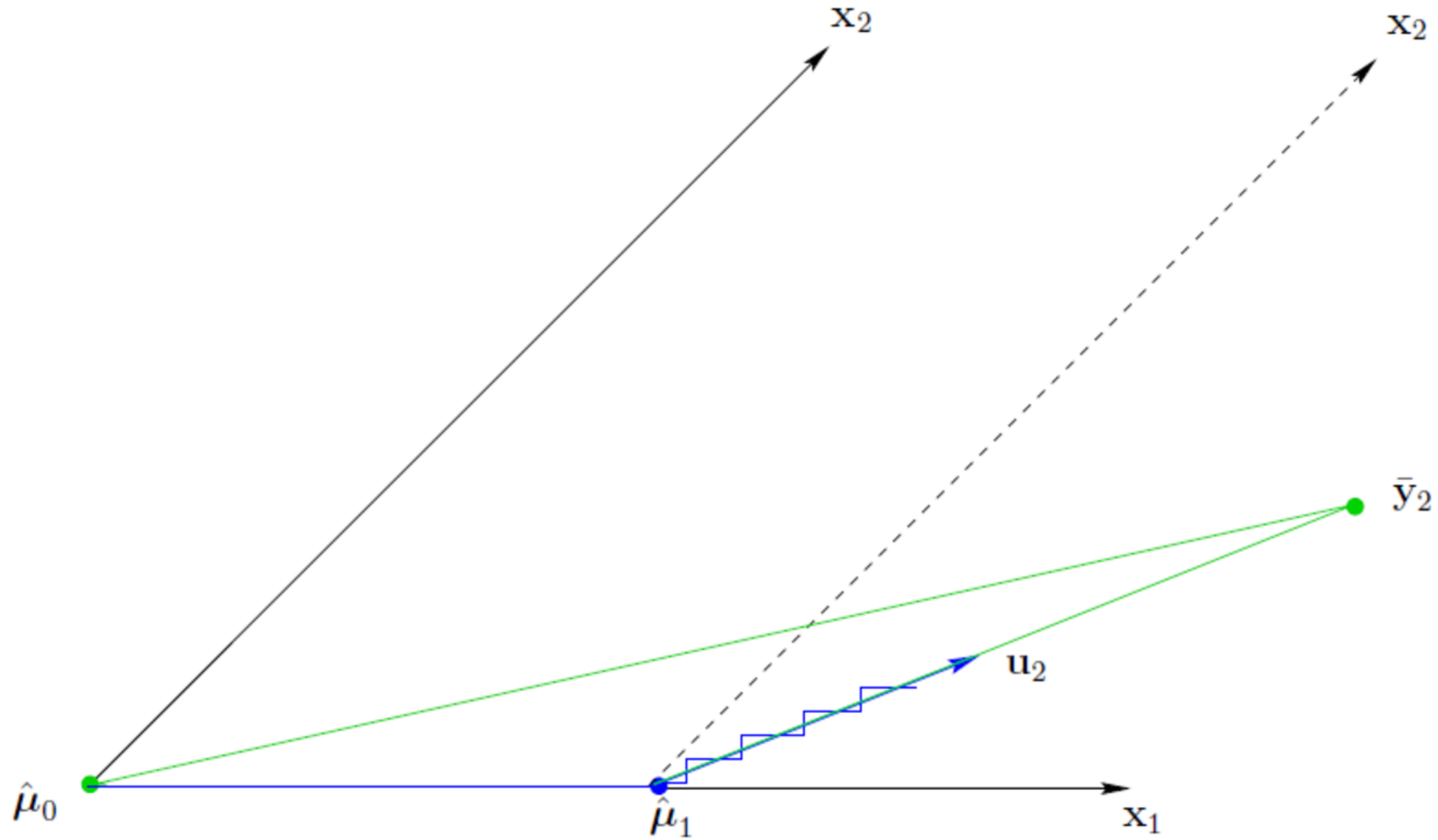
While $\mathbf{w}$ not converged do

1. pick a feature index I
2. update $\mathbf{w}_i$ to $\arg \min_{\mathbf{w}_i} L(\mathbf{w})$

LARS:

1. Take feature x_i that has the highest absolute correlation with y
2. Introduce coefficient β_1 as a multiplier for x_i and increase it (or decrease, in the case of negative correlation). Stop when some other predictor x_j has as much correlation with $r = y - \hat{y}$ as x_i has
3. Introduce β_2 as a multiplier for $(x_i \pm x_j)$
4. to 2
5. Stopping criterion reached

Least Angle Regression



R^2 — score

A useful metric for regression:

$$R^2 = 1 - \frac{u}{v}$$

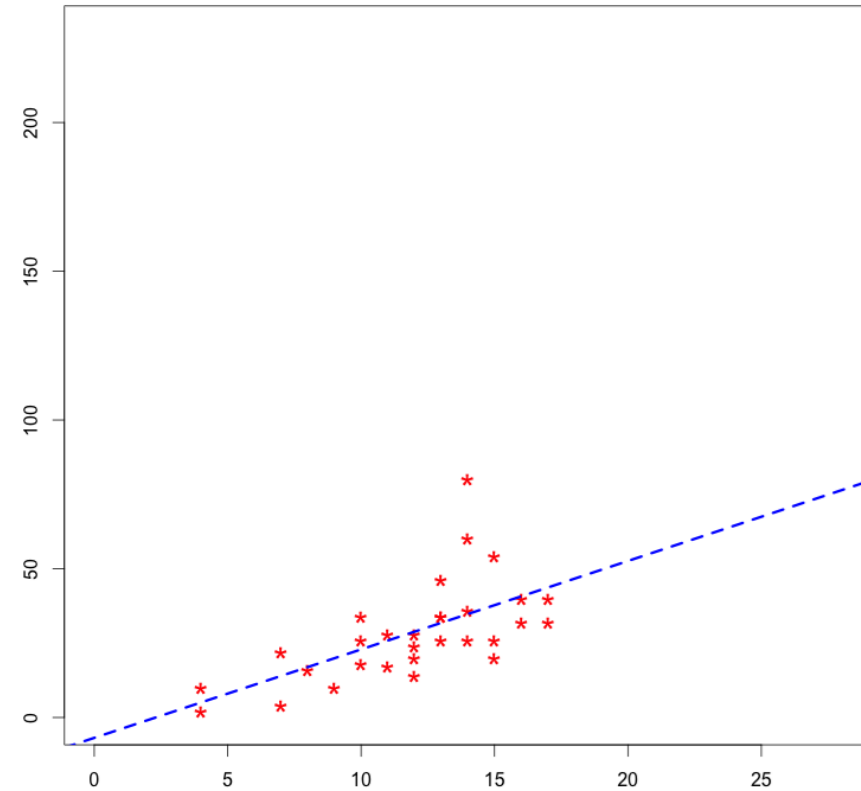
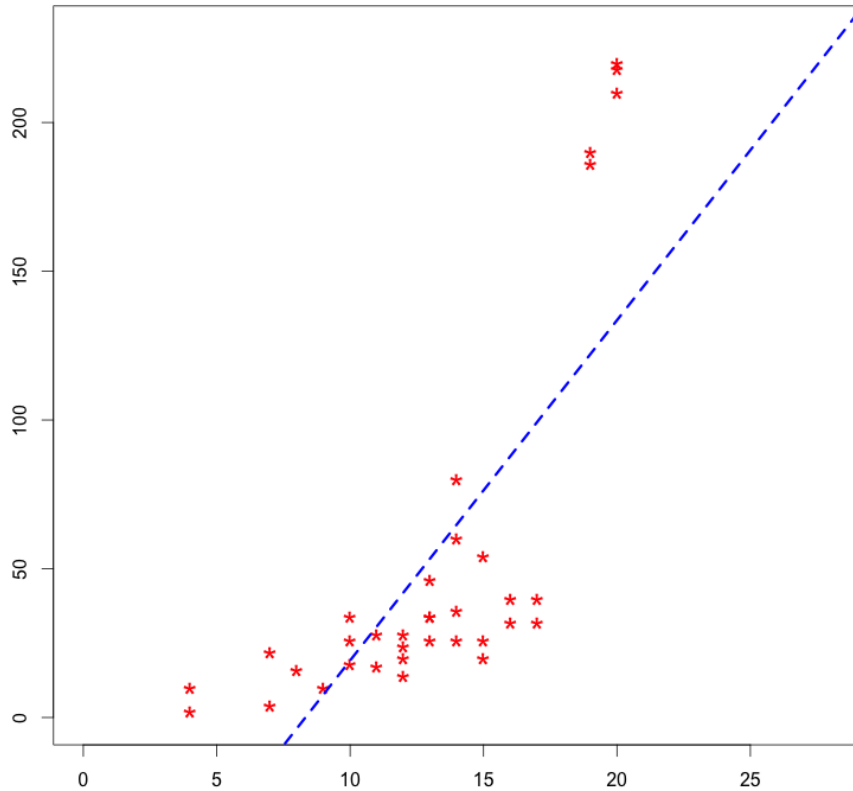
$$u = \sum (h(x_i) - y_i)^2$$

$$v = \sum (\bar{y} - y_i)^2$$

$$\bar{y} = \frac{1}{N} \sum y_i$$

Fighting Outliers

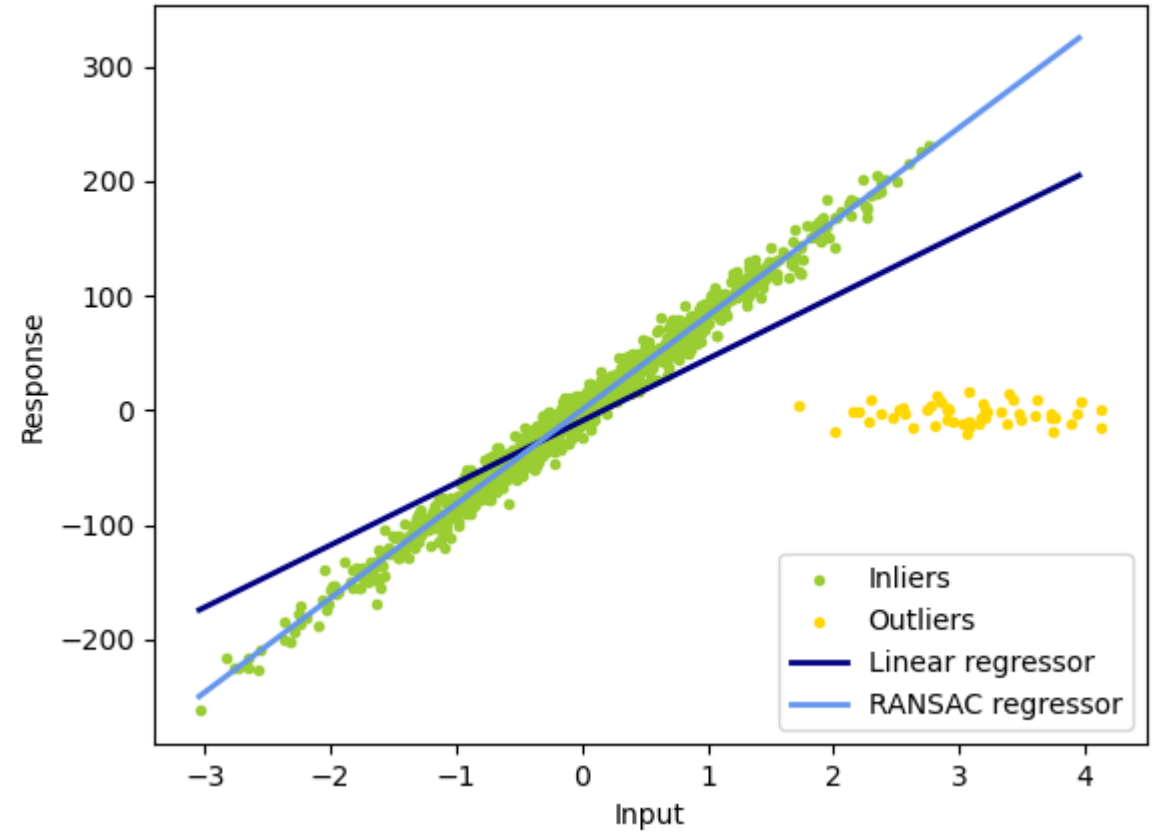
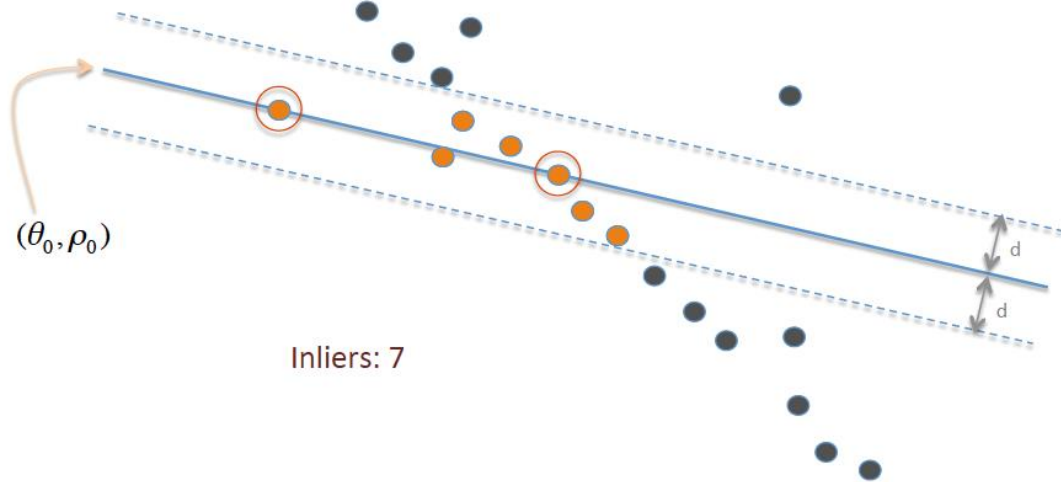
Least squares estimation is sensitive to outliers, so that a few outliers can greatly skew the result.



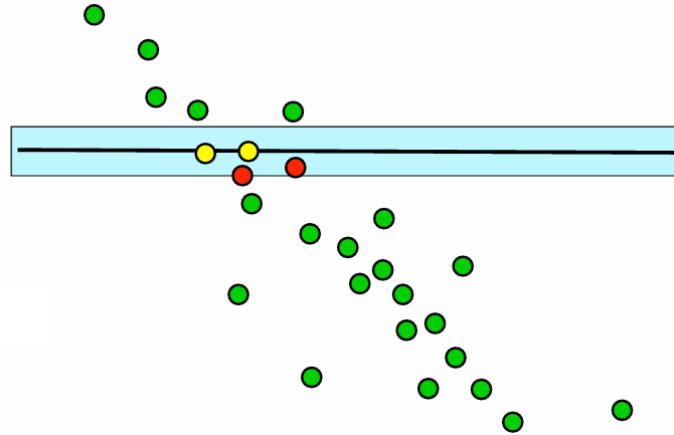
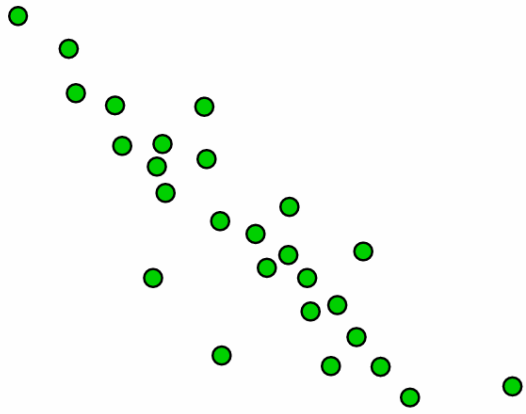
Solution: Estimation methods that are robust to outliers.

RANSAC: RANdom SAmple Consensus

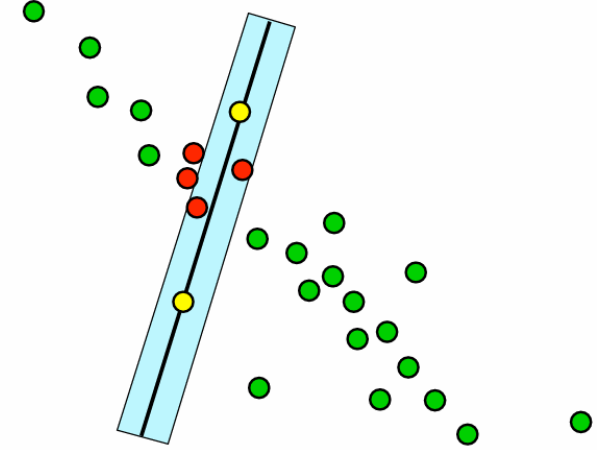
1. Build models on subsets of X
2. Pick the best one in terms of the number of inliers and train a new one on all these inliers.



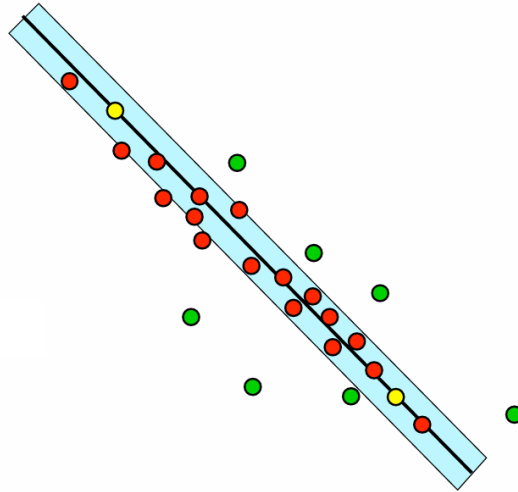
RANSAC: RANdom SAmple Consensus



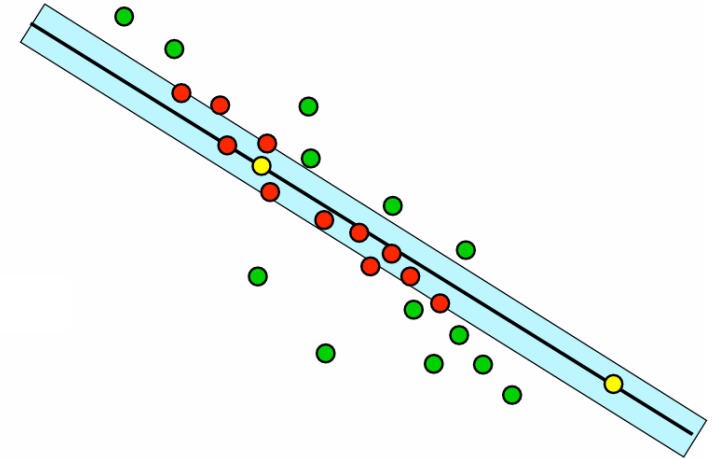
count = 4



count = 6



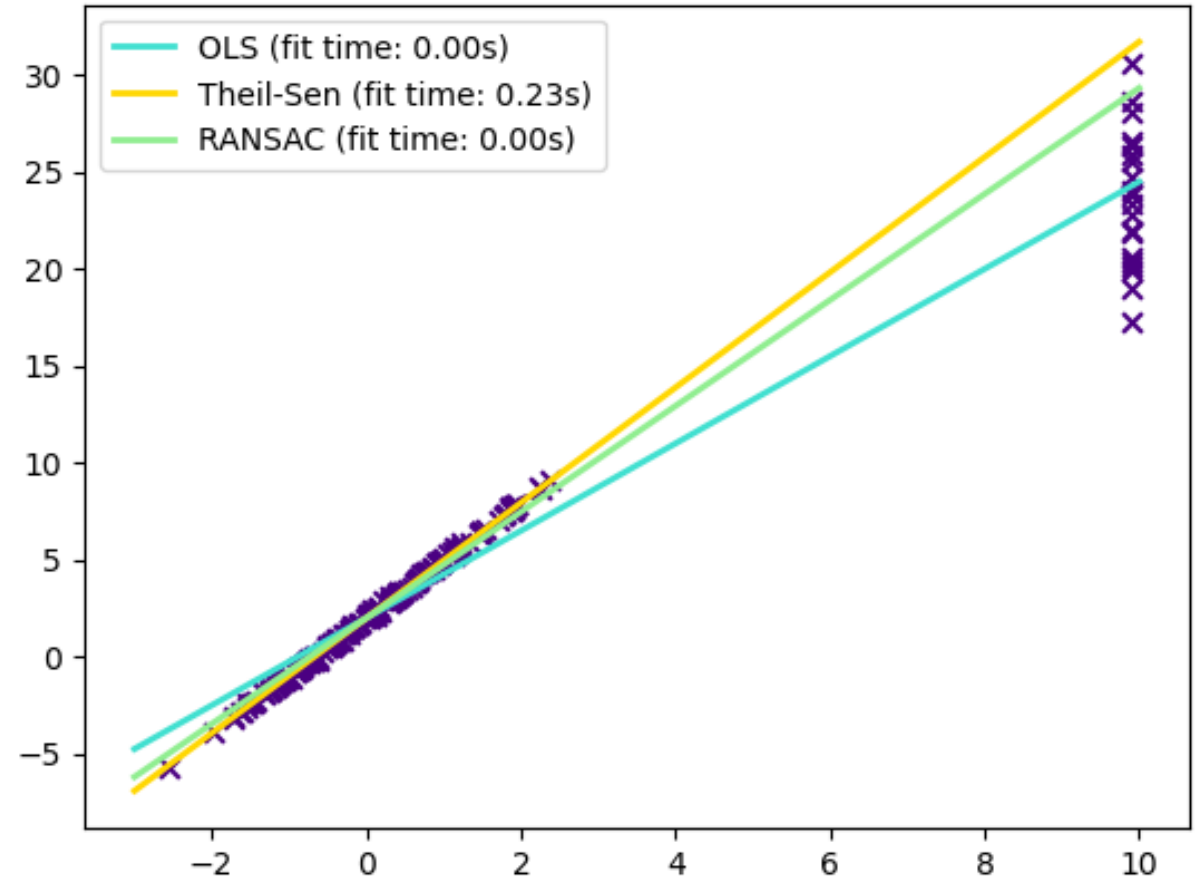
count = 19



count = 13

Theil-Sen Regression

1. Train models on subsets of X .
2. The result is the marginal median of trained models



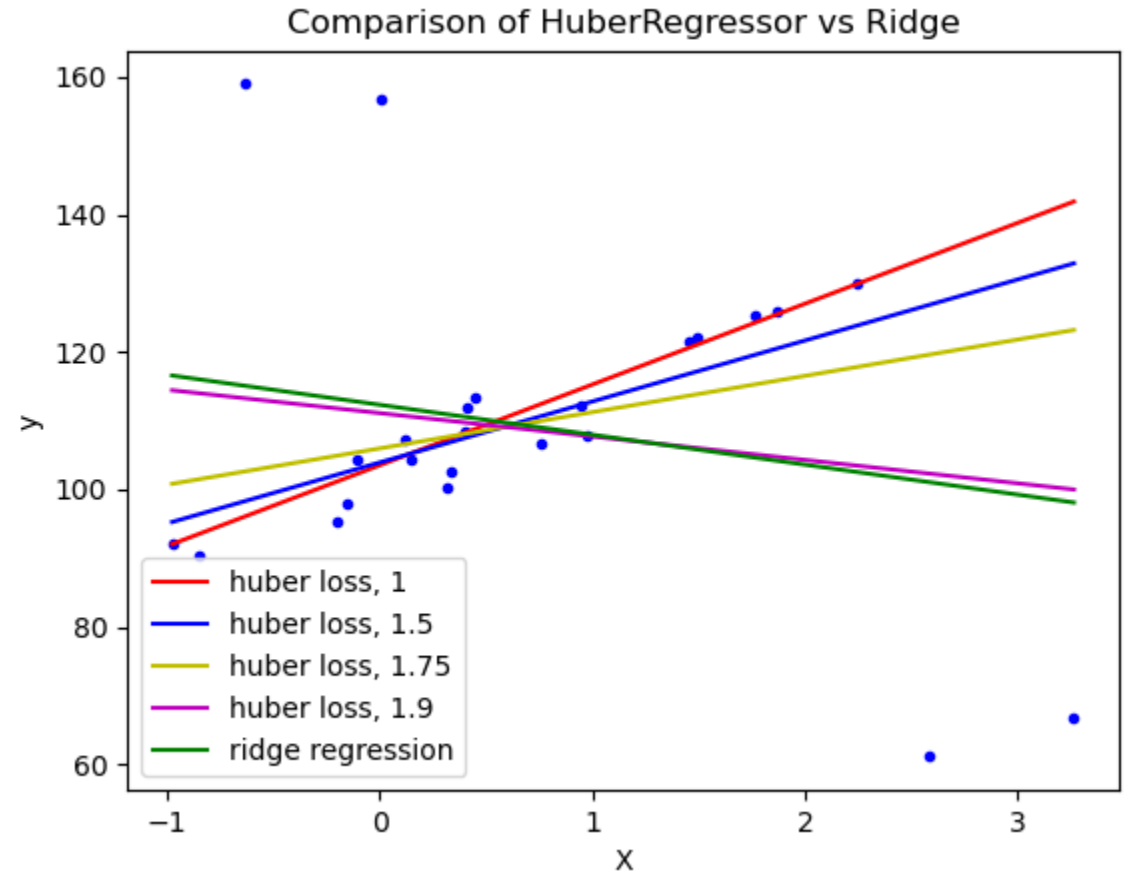
Huber Regressor

Square error for inliers, linear for outliers

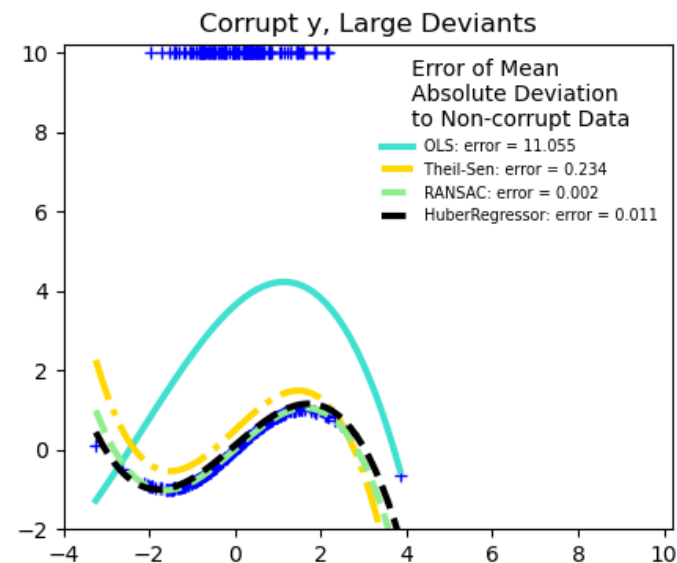
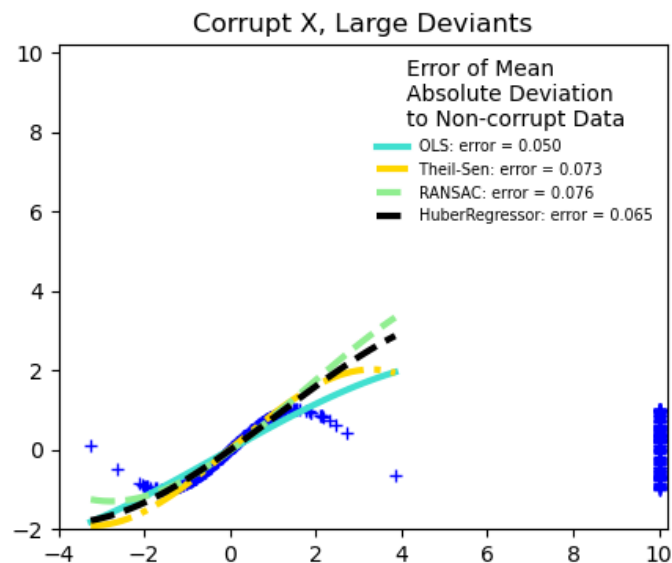
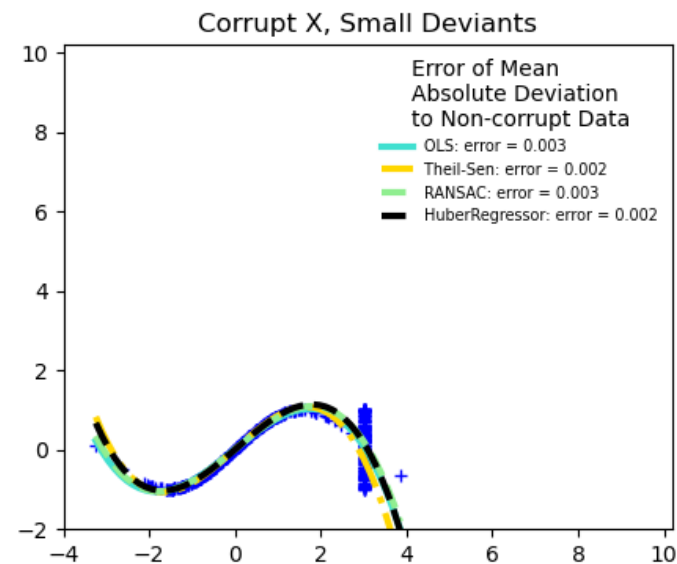
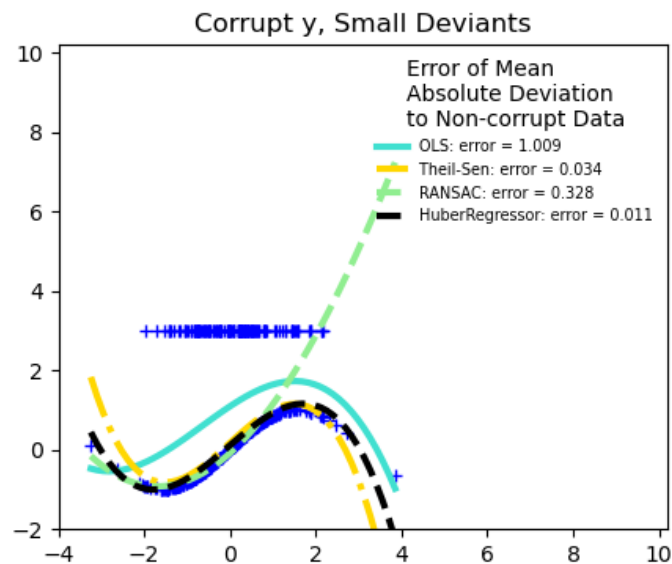
$$\min_{w, \sigma} \sum_{i=1}^N \left(\sigma + H_{\epsilon} \left(\frac{x_i w - y_i}{\sigma} \right) \sigma \right) + \alpha \|w\|_2^2$$

$$H_{\epsilon}(z) = f(x) = \begin{cases} z^2, & \text{if } |z| < \epsilon \\ 2\epsilon|z| - \epsilon^2, & \text{if } |z| \geq \epsilon \end{cases}$$

σ – scaling constant



RANSAC vs. Theil-Sen vs. Huber



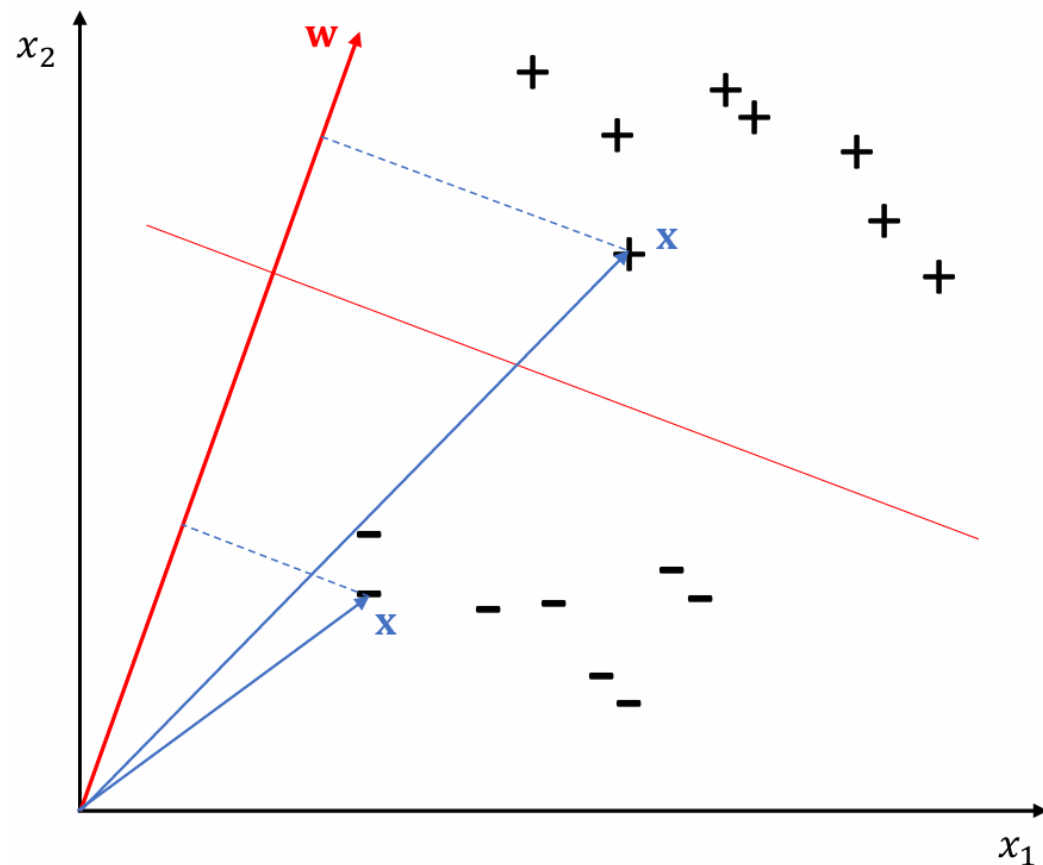
Perceptron

Class 1 if $\sum_{i=1}^d w_i x_i > \text{threshold}$

Class 0 if $\sum_{i=1}^d w_i x_i < \text{threshold}$

$$h(\mathbf{x}) = \text{sign} \left(\left(\sum_{i=1}^d w_i x_i \right) - w_0 \right) = \text{sign} \left(\sum_{i=0}^d w_i x_i \right)$$

$$h(\mathbf{x}) = \text{sign}(\mathbf{w}^T \mathbf{x})$$

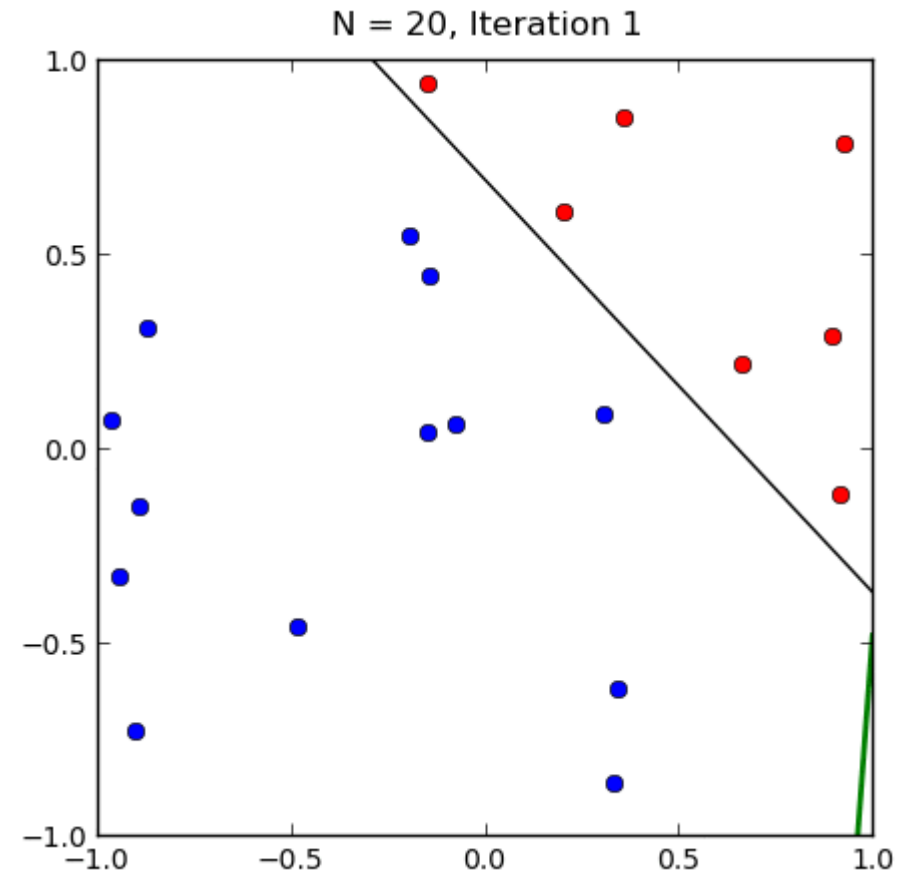


Perceptron Learning

Data: $(\mathbf{x}_1, y_1), \dots, (\mathbf{x}_N, y_N)$

Algorithm:

- Init weights as random
- Pick an $(\mathbf{x}_k, y_k)$ that $y_k \neq \text{sign}(\mathbf{w}_t^T \mathbf{x}_k)$
- Update $\mathbf{w}$: $\mathbf{w}_{t+1} = \mathbf{w}_t + y_k \mathbf{x}_k$



Why Does It Work?

Exercise

1. Show, that $y_k \mathbf{w}_t^T \mathbf{x}_k < 0$
[Hint: $\mathbf{x}_k$ is misclassified by $\mathbf{w}_t$]
2. Show, that $y_k \mathbf{w}_{t+1}^T \mathbf{x}_k > y_k \mathbf{w}_t^T \mathbf{x}_k$
[Hint: Use update step]
3. As far as classifying $\mathbf{x}_k$ is concerned, argue that the move from $\mathbf{w}_t$ to $\mathbf{w}_{t+1}$ is a move “in the right direction”

